

OPERATING SYSTEMS PROJECT

CPU Scheduling | Memory Allocation | Page Replacement

Team Members

Name	Student ID
Kyrillos Ayman Shenouda	2305221
Abdelrahman Hesham Salama	2305239
Amr Khaled Abdelhady	2305224
Hamza Ismail Ahmed	2305349
Abdelrahman Ahmed Abdelrahim	2405158

Academic Year 2024 / 2025

Computer Science Department — Operating Systems Course

1. Project Overview

This report documents the implementation and analysis of fundamental Operating Systems algorithms developed as a team project. The project covers three core areas of operating systems theory: CPU Scheduling, Memory Allocation, and Page Replacement (Virtual Memory Management).

The project was implemented in Python and deployed as an interactive web application using Streamlit, allowing users to input parameters dynamically and visualize results in real time.

1.1 Objectives

- Implement and simulate core OS algorithms from scratch in Python
- Accept all inputs dynamically from the user — no hardcoded values
- Produce step-by-step execution traces and final performance results
- Visualize outputs using tables, Gantt charts, and bar graphs
- Compare chosen algorithms with at least one alternative each
- Analyze performance differences, advantages, disadvantages, and use cases

1.2 Algorithms Implemented

Category	Chosen Algorithm	Compared Against
CPU Scheduling	Round Robin (RR)	FCFS (First Come First Serve)
Memory Allocation	Best Fit	First Fit
Page Replacement	LRU (Least Recently Used)	FIFO (First In First Out)

2. CPU Scheduling — Round Robin

2.1 Algorithm Description

Round Robin (RR) is a preemptive CPU scheduling algorithm designed specifically for time-sharing systems. Each process is assigned a fixed time slice called the time quantum. The CPU executes each process for at most one quantum at a time; if a process does not finish within its quantum, it is moved to the back of the ready queue and the next process is served.

This approach guarantees that no single process monopolizes the CPU, providing fairness and responsiveness across all running processes.

2.2 How It Works

- All arriving processes are placed in a circular ready queue sorted by arrival time.
- The scheduler picks the first process from the queue and executes it for $\min(\text{quantum}, \text{remaining_burst})$ time units.
- If remaining burst time > 0 after execution, the process is re-added to the rear of the queue.
- New processes that arrive during a quantum are added to the queue before the current process is re-queued.
- The loop continues until all processes complete.

2.3 Input Parameters

- Number of processes (n)
- Arrival Time for each process
- Burst Time for each process
- Time Quantum (q)

2.4 Output & Results

The following sample run uses 3 processes with a time quantum of 2:

 Operating Systems Project CPU Scheduling  Memory Allocation  Page Replacement  Comparison & Analysis **Round Robin CPU Scheduling**

Number of Processes

3  

Time Quantum

2   **Process Details**

Process

Arrival Time

Burst Time

P1

0  2  

P2

2  30  

P3

1  50  *Figure 1: Round Robin CPU Scheduling — Input form (3 processes, quantum = 2)*

Process	Arrival	Burst	Completion	Turnaround	Waiting
P1	0	2	2	2	0
P2	2	30	62	60	30
P3	1	50	82	81	31

Metric	Value
Average Waiting Time	20.33
Average Turnaround Time	47.67

The Gantt chart (produced by the application) shows P1 executing first from time 0–2, followed by alternating P2 and P3 slices of 2 time units each until both complete at times 62 and 82 respectively.

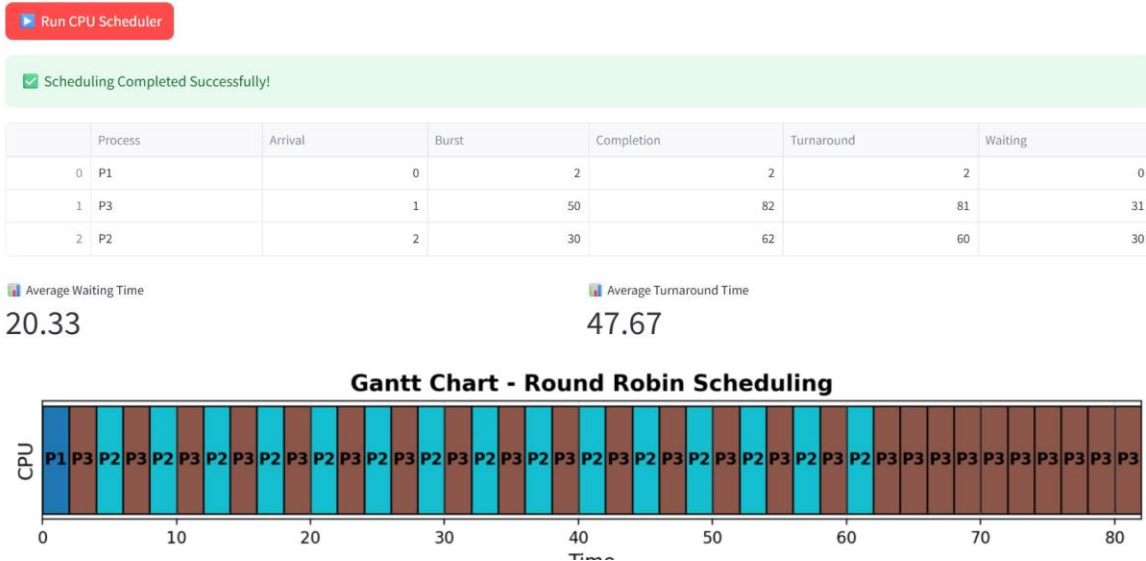


Figure 2: Round Robin results table and Gantt chart (Avg WT = 20.33, Avg TAT = 47.67)

2.5 Formulas

- Turnaround Time (TAT) = Completion Time – Arrival Time
- Waiting Time (WT) = Turnaround Time – Burst Time
- Average WT = Sum of all Waiting Times / n
- Average TAT = Sum of all Turnaround Times / n

2.6 Round Robin vs. FCFS — Comparison

Feature	Round Robin	FCFS
Type	Preemptive	Non-Preemptive
Waiting Time	Lower (avg)	Higher for short jobs
Response Time	Fast & predictable	Slow if large jobs arrive first
Fairness	High — equal CPU slices	Low — FIFO order only
Context Switches	More frequent	Fewer
Convoy Effect	Not present	Present
Best Use Case	Time-sharing / Interactive	Batch processing systems

Advantages of Round Robin

- Provides fair CPU time distribution across all processes
- No starvation — every process eventually gets the CPU

- Excellent response time in interactive and real-time systems
- Easy to implement with a simple circular queue

Disadvantages of Round Robin

- Performance is highly sensitive to the choice of quantum size
- High number of context switches increases overhead
- Larger average turnaround time compared to SJF for CPU-bound jobs

3. Memory Allocation — Best Fit

3.1 Algorithm Description

Best Fit is a dynamic memory allocation strategy in which, when a new process requests memory, the allocator searches the entire list of free memory blocks and assigns the smallest block that is still large enough to satisfy the request. The goal is to minimize wasted (internal) fragmentation per allocation by finding the tightest fit.

3.2 How It Works

- The system maintains a list of free memory blocks, each with a known size.
- For each incoming process, iterate over all free blocks and find those with size $\geq$ process size.
- Among eligible blocks, select the one with the minimum size (closest fit).
- Allocate the process to that block; internal fragmentation = block size – process size.
- The block's remaining space is reduced accordingly.
- If no block fits, the process is marked 'Not Allocated'.

3.3 Input Parameters

- Number of memory blocks and their individual sizes (KB)
- Number of processes and their memory requirements (KB)

3.4 Output & Results

Sample run — Memory Blocks: [100, 500, 200, 300, 600] KB | Processes: [212, 417, 112, 426] KB

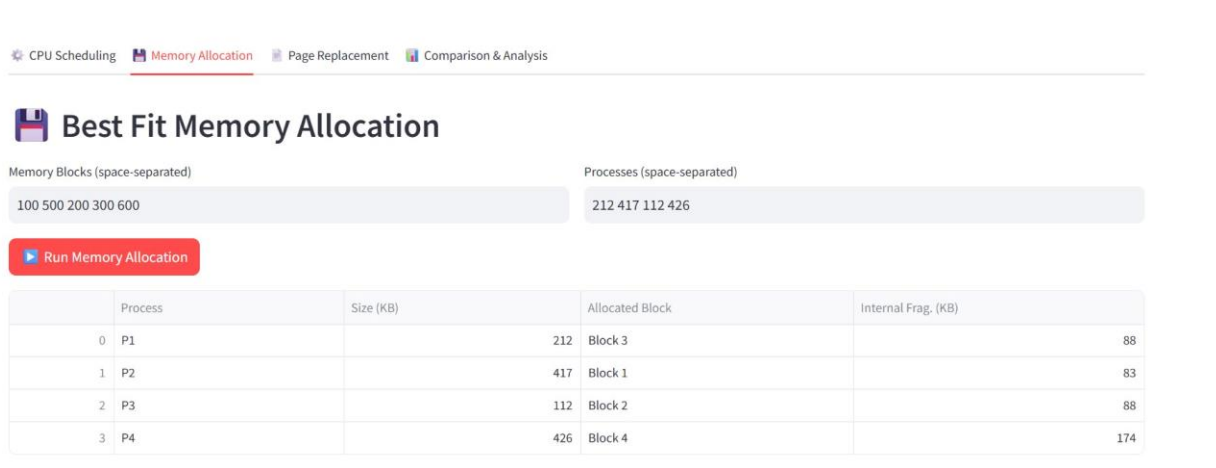


Figure 3: Best Fit Memory Allocation — input and allocation results table

Process	Size (KB)	Allocated Block	Internal Frag. (KB)
P1	212	Block 3 (300 KB)	88
P2	417	Block 1 (500 KB)	83
P3	112	Block 2 (200 KB)	88
P4	426	Block 4 (600 KB)	174

Metric	Value
Total Internal Fragmentation	433 KB
External Fragmentation	533 KB

All four processes were successfully allocated. The bar chart produced by the application confirms all processes are in the 'Allocated' state (shown in green).

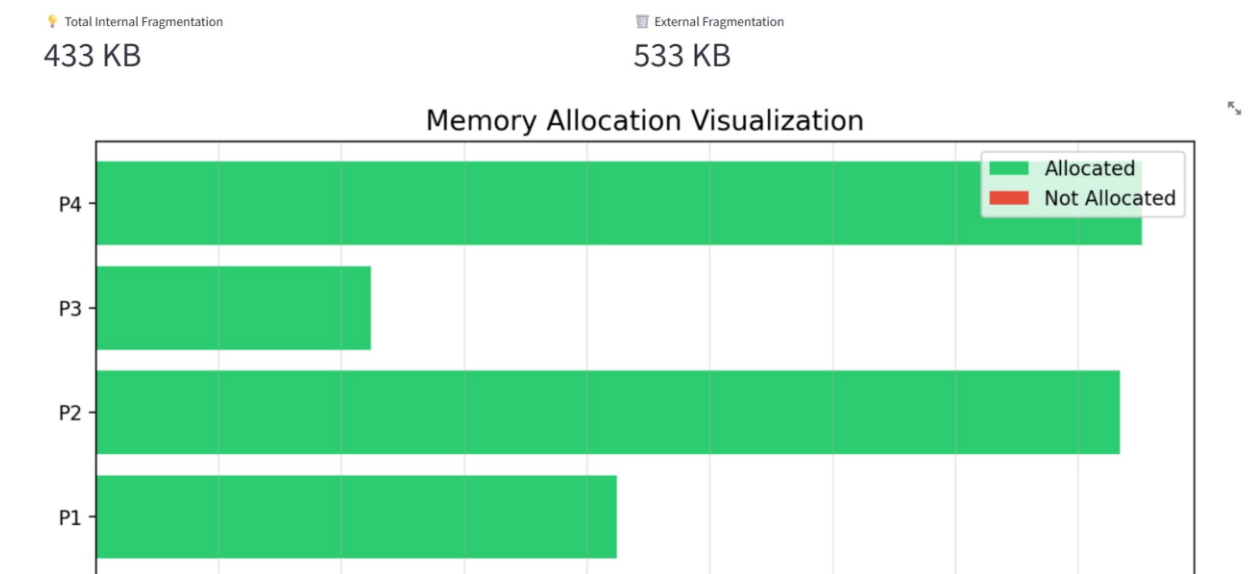


Figure 4: Memory Allocation visualization — all 4 processes allocated (green bars)

3.5 Fragmentation Types

- Internal Fragmentation: Wasted space inside an allocated block (block size – process size).
- External Fragmentation: Total remaining free space in all blocks after allocation.

3.6 Best Fit vs. First Fit — Comparison

Feature	Best Fit	First Fit
Search Method	Entire free list	First suitable block
Speed	Slower ($O(n)$ always)	Faster on average
Internal Fragmentation	Lower	Higher
External Fragmentation	More tiny holes	Fewer but larger holes
Memory Utilization	Better	Moderate
Best Use Case	Memory-constrained systems	General-purpose OS

Advantages of Best Fit

- Minimizes wasted space per allocation
- Better overall memory utilization than First Fit
- Reduces the likelihood of large blocks being split unnecessarily

Disadvantages of Best Fit

- Requires scanning the entire free list — slower than First Fit
- Can create many tiny unusable memory fragments over time
- Compaction may eventually be needed to merge small free fragments

4. Page Replacement — LRU

4.1 Algorithm Description

Least Recently Used (LRU) is a page replacement policy used in virtual memory management. When a page fault occurs and all frames are occupied, LRU evicts the page that has not been referenced for the longest time. The intuition is that pages used recently are likely to be used again soon (temporal locality), so the oldest unused page is the least valuable to keep.

4.2 How It Works

- Maintain a set of loaded frames and a timestamp (last_used) for each page.
- For each page reference: if the page is already in frames (HIT), update its timestamp.
- If the page is not in frames (FAULT): increment fault counter.
 - If free frames exist, load the page into an empty frame.
 - Otherwise, find the page with the oldest timestamp and replace it.
- Record frame state after each reference for the step-by-step trace.

4.3 Input Parameters

- Page Reference String (space-separated page numbers)
- Number of Frames

4.4 Output & Results

Sample run — Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2 | Frames: 3

 **LRU Page Replacement Algorithm**

Page Reference String

7 0 1 2 0 3 0 4 2 3 0 3 2

Number of Frames

3

 Run LRU Simulation

Figure 5: LRU Page Replacement — input form (reference string and frame count)

#	Time	Page	Frame 0	Frame 1	Frame 2	Status
0	1	7	7	—	—	Fault
1	2	0	7	0	—	Fault
2	3	1	7	0	1	Fault
3	4	2	2	0	1	Fault

#	Time	Page	Frame 0	Frame 1	Frame 2	Status
4	5	0	2	0	1	Hit
5	6	3	2	0	3	Fault
6	7	0	2	0	3	Hit
7	8	4	4	0	3	Fault
8	9	2	4	0	2	Fault
9	10	3	4	3	2	Fault
10	11	0	0	3	2	Fault
11	12	3	0	3	2	Hit
12	13	2	0	3	2	Hit

Metric	Value	
Total Page Faults	9	
Total Hits	4	
Fault Rate	69.2%	

Q						
	Time	Page	Frame 0	Frame 1	Frame 2	Status
0	1	7	7	-	-	✔ Fault
1	2	0		0	-	✔ Fault
2	3	1		0	1	✔ Fault
3	4	2		0	1	✔ Fault
4	5	0		0	1	✖ Hit
5	6	3		0	3	✔ Fault
6	7	0		0	3	✖ Hit
7	8	4		0	3	✔ Fault
8	9	2		0	2	✔ Fault
9	10	3		3	2	✔ Fault
10	11	0		3	2	✔ Fault
11	12	3		3	2	✖ Hit
12	13	2		3	2	✖ Hit

Figure 6: LRU step-by-step frame trace — 9 faults, 4 hits (13 references, 3 frames)

4.5 LRU vs. FIFO — Comparison

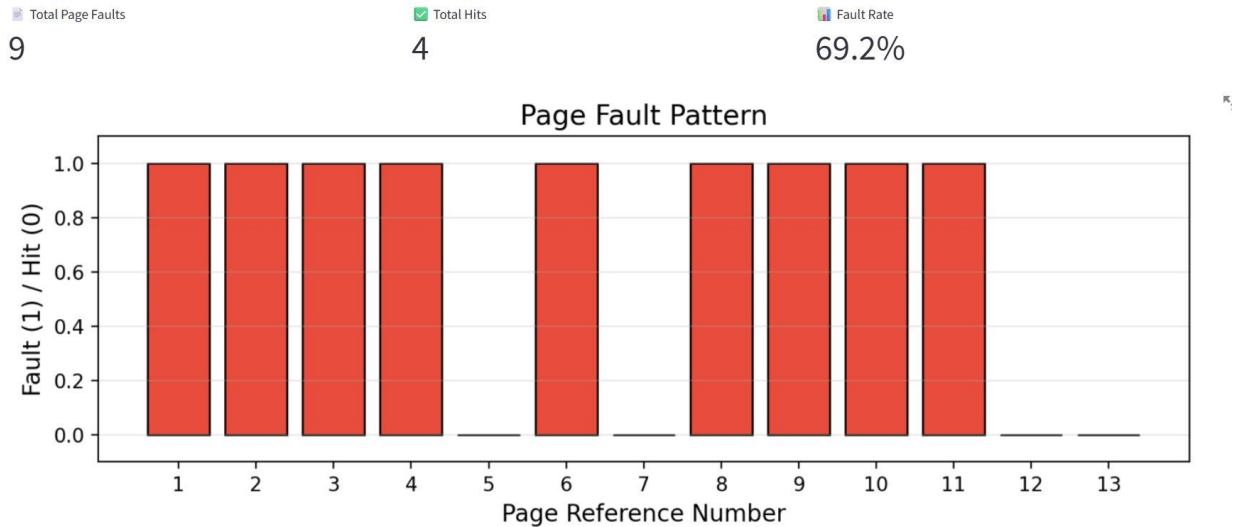


Figure 7: Page Fault Pattern chart — red bars = faults, empty = hits (fault rate 69.2%)

Feature	LRU	FIFO
Replacement Policy	Least recently used page	Oldest loaded page
Page Faults	Lower (better performance)	Higher on average
Locality Awareness	Yes — uses temporal locality	No — ignores usage pattern
Belady's Anomaly	Not subject to it	Can suffer from it
Implementation	Complex (timestamp tracking)	Simple (queue)
Hardware Support	Often needed	Not required
Best Use Case	General-purpose OS kernels	Simple embedded systems

Advantages of LRU

- Closely approximates the theoretically optimal (OPT) replacement policy
- Exploits temporal locality — recently used pages are kept in memory
- Not susceptible to Belady's anomaly
- Consistently achieves fewer page faults than FIFO in real workloads

Disadvantages of LRU

- More complex to implement — requires timestamp or counter per page
- May need hardware support (reference bits or dedicated registers)
- Slightly higher per-access overhead compared to FIFO

5. Implementation Details

5.1 Project Structure

File / Module	Description
main.py	CLI entry point — menu-driven interface
cpu_round_robin.py	Round Robin scheduler (CLI version)
memory_best_fit.py	Best Fit allocator (CLI version)
page_replacement_lru.py	LRU page replacement (CLI version)
comparison.py	Comparison & analysis tables (CLI)
app.py (Streamlit)	Full interactive web UI — all modules
requirements.txt	Python dependencies

5.2 Libraries Used

- `tabulate` — formatted terminal tables for CLI output
- `matplotlib` — bar charts and Gantt chart visualization
- `pandas` — DataFrame creation for Streamlit data tables
- `numpy` — color map generation for Gantt chart process colors
- `streamlit` — interactive web application framework
- `collections.deque` — efficient queue operations in Round Robin scheduler

5.3 Key Design Decisions

- All inputs accepted dynamically at runtime — zero hardcoded values.
- CLI and Streamlit versions share the same algorithm logic.
- LRU uses a dictionary of timestamps (`last_used`) for $O(n)$ replacement; the page with the minimum timestamp among current frames is evicted.
- Round Robin re-queues the current process before adding newly arrived processes to ensure correct ordering.
- Best Fit reduces the allocated block's remaining size after each allocation, allowing partial reuse of large blocks for future processes.

6. Conclusion

This project provided hands-on experience implementing three fundamental Operating Systems algorithms that underpin real-world operating systems design. By building each algorithm from scratch and comparing it against an alternative, the team developed a thorough understanding of the trade-offs involved in OS resource management.

6.1 Key Findings

- Round Robin CPU Scheduling offers the best fairness and response time among preemptive schedulers. The quantum size is the critical tuning parameter — a small quantum improves interactivity but increases context-switch overhead, while a large quantum approaches FCFS behaviour.
- Best Fit Memory Allocation consistently minimizes internal fragmentation per allocation. However, it can produce many small unusable holes over time, and its $O(n)$ search cost is higher than First Fit for workloads with frequent allocations.
- LRU Page Replacement delivers significantly fewer page faults than FIFO by leveraging temporal locality. While FIFO is simpler to implement, LRU's superior hit ratio makes it the practical choice for general-purpose operating systems.

6.2 Summary Table

Criterion	Round Robin	Best Fit	LRU
Primary Goal	Fairness	Memory efficiency	Reduce page faults
Main Strength	No starvation	Low int. fragmentation	Temporal locality
Main Weakness	Context switches	Slow search	Complex tracking
Best Environment	Interactive / Time-sharing	Memory-constrained	General-purpose OS

All project requirements were met: algorithms were implemented in Python, inputs are fully dynamic, outputs include step-by-step traces and visual charts, and each algorithm is compared against a documented alternative. The Streamlit web application provides an accessible, interactive interface for demonstrating all three algorithms simultaneously.

7. References

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
- Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson.
- Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>
- Streamlit Inc. Streamlit Documentation. <https://docs.streamlit.io/>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.
- Tabulate library documentation. <https://pypi.org/project/tabulate/>